Rapport de projet Master 1 Informatique

Décodage sous contraintes pour les LLMs

Analyse, Reproduction et Complétion des résultats du *JSONSchemaBench*

Étudiant : *ARDJAL YOUNES*
Étudiant : *MANCER SAMY*
Encadrant : *Mohamed-Amine Baazizi, Nour ElHouda Ben Ali*
Institution : *SORBONNE UNIVERSITE*

Date : *10 octobre 2026*

Abstract

Ce rapport évalue et reproduit les résultats du benchmark *JSONSchemaBench* (Geng et al., 2025) pour les frameworks de décodage sous contraintes **Outlines** et **Guidance**, dans le cadre de la génération de sorties structurées JSON par les LLMs.

Table des matières

1 Contexte et objectifs du travail **2**
1.1 L'essor des sorties structurées dans les systèmes à base de LLMs 2
1.2 JSON Schema : spécifier la structure attendue 2
1.3 Le décodage contraint comme mécanisme de contrôle 3
1.4 Le *JSONSchemaBench* : un benchmark central 4
1.5 Métriques d'évaluation 4
1.5.1 Efficacité computationnelle 4
1.5.2 Couverture et conformité 5
1.6 Objectifs du travail 5

2 Constrained Decoding et JSON Schema **7**
2.1 Mécanisme du décodage sous contraintes 7
2.1.1 Génération non contrainte 7
2.1.2 Limite du décodage standard 7
2.1.3 Formulation du décodage sous contraintes 7
2.1.4 Résolution locale par masquage des logits 8
2.2 JSON Schema comme grammaire formelle 9
2.2.1 Exemple simple : un objet utilisateur 9
2.2.2 Exemple composé : un objet projet 10
2.3 Écosystème des frameworks de constrained decoding 12

3 Contributions expérimentales **15**
3.1 Environnement d'expérimentation et ressources 15
3.2 Évaluation empirique : le *JSONSchemaBench* et ses pipelines 16
3.2.1 La base de données *JSONSchemaBench* 16
3.2.2 Pipeline d'évaluation de la couverture 17
3.3 Expérimentations et Résultats 20
3.3.1 Résultat de l'article (JSONSchemaBench) 20
3.3.2 Notre campagne d'expérimentation 21
3.4 Analyse comparative et apports 27
3.4.1 Nécessité d'un framework de *Constrained Decoding* 27
3.4.2 Synthèse comparative : Outlines vs Guidance 27
3.4.3 Tableau récapitulatif 28

4 Perspectives **29**
4.1 Travaux non réalisés par manque de temps 29
4.1.1 Extension à XGrammar et comparaison à trois 29
4.1.2 Évaluation du Pipeline A 29
4.1.3 Modélisation explicable des causes d'échec 29
4.1.4 Explorer le décodage contraint sous d'autres formes 30

5 Conclusion des expérimentations **31**

Chapitre 1

Contexte et objectifs du travail

1.1 L'essor des sorties structurées dans les systèmes à base de LLMs

Les grands modèles de langage (LLMs) ont montré une capacité remarquable à produire des contenus cohérents et plausibles dans une grande variété de tâches. Cette progression a favorisé leur intégration croissante dans des systèmes logiciels réels, où ils ne sont plus seulement mobilisés pour la génération de texte libre, mais aussi pour la production de données structurées directement exploitables par des programmes.

Dans ce cadre, la sortie du modèle n'est plus uniquement destinée à un lecteur humain. Elle peut devoir être analysée automatiquement, transmise à une API, interprétée comme un appel de fonction ou encore insérée dans une base de données. Cette évolution impose deux exigences fondamentales.

- **Validité syntaxique.**
  La sortie doit respecter un format formel précis, le plus souvent JSON. Une erreur apparemment mineure accolade manquante, guillemet non fermé ou virgule mal placée suffit à rendre la sortie non parsable et peut, par conséquent, provoquer l'échec de toute la chaîne de traitement.
- **Conformité au schéma.**
  Au-delà de la syntaxe, la sortie doit satisfaire un contrat de données explicite : présence des champs obligatoires, respect des types attendus, contraintes sur les valeurs admissibles et organisation correcte des objets ou tableaux imbriqués. Ainsi, une sortie JSON syntaxiquement valide peut néanmoins rester inutilisable si elle ne respecte pas le schéma cible.

Or, un LLM utilisé en génération libre demeure un modèle probabiliste. Même lorsqu'il reçoit des consignes précises, il peut produire une réponse partiellement structurée, mêler texte naturel et JSON, omettre des champs requis ou générer des valeurs incompatibles avec les types attendus. En pratique, ces écarts imposent souvent l'ajout d'étapes de post-traitement, de validation ou de relance automatique, ce qui augmente la complexité des systèmes, leur coût d'exécution et leur fragilité opérationnelle.

Dans cette perspective, les sorties structurées constituent désormais un enjeu central de l'ingénierie des LLMs. L'objectif n'est plus seulement d'inciter le modèle à suivre un format donné par le prompt, mais de contrôler plus finement le processus de génération afin d'augmenter la probabilité que la sortie soit, dès sa production, directement exploitable par le système aval.

1.2 JSON Schema : spécifier la structure attendue

Le format le plus couramment mobilisé pour représenter des sorties structurées est **JSON** (*JavaScript Object Notation*). Sa simplicité, sa lisibilité et son support natif dans la plupart des

langages de programmation en font un standard de fait pour l'échange de données entre services web, API et systèmes logiciels.

Toutefois, JSON ne décrit que la syntaxe générale d'un document. À lui seul, il ne permet pas d'exprimer quels champs doivent être présents, quels types sont attendus, ni quelles contraintes doivent porter sur les valeurs. Pour répondre à ce besoin, on utilise **JSON Schema**, un langage de spécification conçu pour décrire de manière formelle la structure attendue d'un document JSON.

JSON Schema permet notamment de définir :

- les types des champs, par exemple `string`, `integer`, `array` ou `object` ;
- les champs obligatoires au moyen de la clause `required` ;
- des contraintes de valeur telles que `minimum`, `maximum`, `maxLength`, `pattern` ou `enum` ;
- la structure d'objets et de tableaux imbriqués via `properties` et `items` ;
- des opérateurs logiques comme `anyOf`, `oneOf`, `allOf` et `not`.

Le langage offre également des mécanismes de référence permettant de réutiliser des fragments de schéma, de factoriser les définitions et d'exprimer des structures potentiellement récursives. Cette expressivité est particulièrement utile lorsque les objets générés comportent plusieurs niveaux d'imbrication ou des contraintes combinatoires non triviales.

Historiquement, JSON Schema a été introduit pour décrire et valider la structure de données publiées ou échangées sur le web. Il est également employé dans certains systèmes de stockage documentaire ou NoSQL afin de restreindre la forme des données admissibles. Dans le contexte des LLMs, son rôle est plus spécifique : il constitue un **contrat formel** entre le système appelant et le modèle. Générer une sortie conforme ne signifie donc pas seulement produire un JSON bien formé, mais produire un document dont la structure et les valeurs satisfont effectivement l'ensemble des contraintes exprimées par le schéma.

1.3 Le décodage contraint comme mécanisme de contrôle

Pour contraindre les LLMs à produire des données structurées, plusieurs approches ont été proposées. Parmi elles, la plus répandue est le **décodage contraint** (*constrained decoding*), qui consiste à intervenir directement pendant la phase de génération plutôt qu'à corriger la sortie a posteriori.

À chaque étape de décodage, le modèle attribue une probabilité à l'ensemble des jetons (*tokens*) susceptibles d'être produits. Le principe du décodage contraint est de restreindre cet ensemble aux seuls jetons compatibles avec une spécification cible, par exemple une grammaire formelle ou un schéma. Dans le cas qui nous intéresse, un `JSON Schema` est converti en une représentation interne (telle qu'une `grammaire hors contexte`, un `automate` ou une `machine à états`) permettant au framework de déterminer, à chaque instant, quels jetons restent légalement autorisés.

L'objectif est ainsi d'orienter la génération vers une sortie JSON à la fois syntaxiquement valide et conforme au schéma demandé. En pratique, cette propriété n'est toutefois pas garantie de manière uniforme dans tous les cas.

- `JSON Schema` est un langage expressif, comportant des mécanismes dont la traduction peut être coûteuse ou délicate, notamment les combinateurs logiques, la négation, les références, ainsi que certaines contraintes portant sur les objets ou les tableaux.
- Lorsqu'un framework ne prend pas intégralement en charge un schéma, ou lorsqu'il recourt à des approximations pour certaines contraintes, la sortie générée peut rester partiellement non conforme malgré l'usage d'un mécanisme de décodage contraint.

Ces approches soulèvent également des enjeux de **performance**. Parce qu'elles interviennent directement dans la boucle de génération du modèle, elles peuvent :

- introduire un coût initial de compilation ou de préparation du schéma (on va appeler ceci textitprétraitement) ;
- ralentir le calcul de l'ensemble des jetons autorisés à chaque étape (*overhead*) ;
- provoquer des erreurs, des blocages ou des dégradations de robustesse lorsque les schémas deviennent particulièrement complexes.

Le décodage contraint doit donc être évalué selon une double perspective : d'une part, sa capacité à garantir la **conformité** des sorties ; d'autre part, le **coût computationnel** qu'il introduit au moment de la génération.

1.4 Le *JSONSchemaBench* : un benchmark central

Malgré l'importance industrielle du décodage contraint, l'écosystème des frameworks disponibles (notamment *Outlines*, *Guidance*, *XGrammar* ou *LlamaCpp*) a longtemps souffert d'un manque de comparaison rigoureuse et systématique. Chaque framework tendait à rapporter ses propres résultats sur des jeux de test ad hoc, avec des protocoles expérimentaux hétérogènes, rendant difficile toute comparaison objective de leurs performances et de leur niveau réel de conformité.

Geng et al. (2025) ont proposé une réponse à cette limite avec *JSONSchemaBench*, un benchmark à visée industrielle rassemblant près de **9 558 schémas JSON** issus de cas d'usage réels, notamment de dépôts GitHub, de fichiers de configuration Kubernetes, d'API commerciales et de jeux de données liés à l'appel de fonctions. Ce corpus couvre un spectre de complexité particulièrement large, allant de schémas simples à des structures fortement imbriquées et riches en contraintes.

Le benchmark évalue les frameworks selon trois axes complémentaires :

- **l'efficacité computationnelle**, à travers des mesures telles que le temps de compilation du schéma ou la latence de génération ;
- **la couverture et la conformité**, c'est-à-dire la capacité à produire des sorties valides au regard du schéma demandé ;
- **la qualité sémantique**, entendue comme la préservation de la capacité du LLM à raisonner et à produire un contenu pertinent malgré les contraintes imposées au décodage.

La section suivante précise ces trois axes et les métriques utilisées dans le benchmark, afin de clarifier le cadre d'évaluation mobilisé dans la suite du rapport.

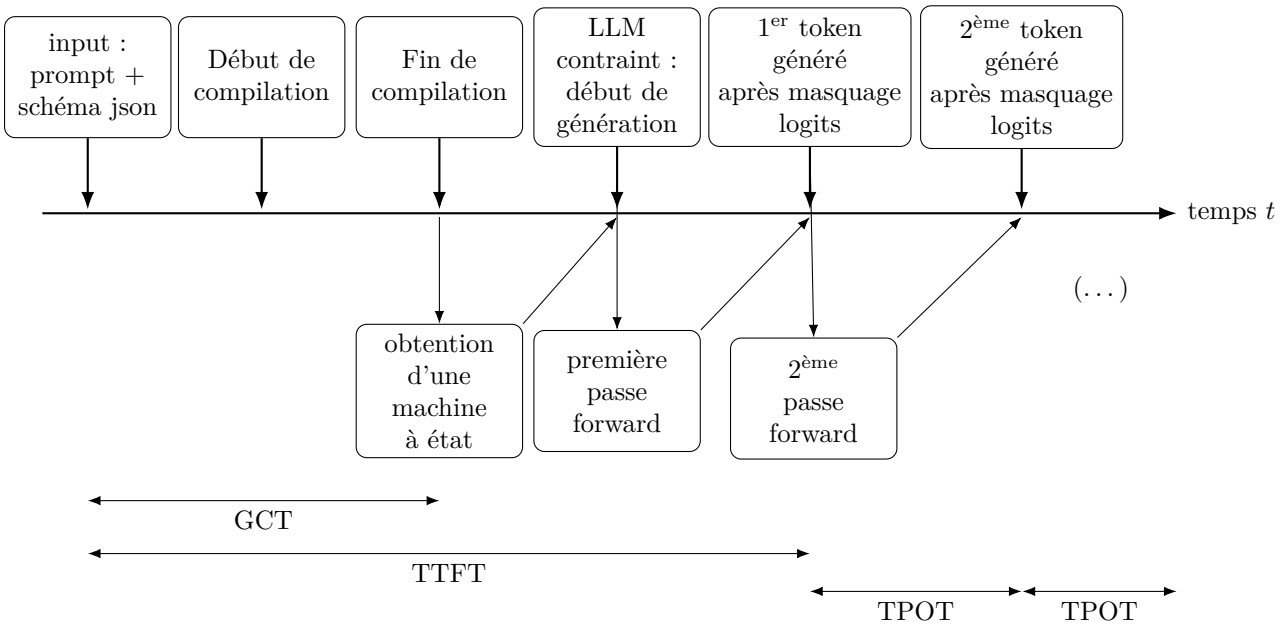
1.5 Métriques d'évaluation

L'évaluation d'un framework de décodage contraint repose sur trois axes et sur les métriques associées, telles qu'elles sont formalisées dans le *JSONSchemaBench*. L'enjeu n'est pas seulement de mesurer la vitesse du framework, mais aussi de caractériser la fidélité de la contrainte compilée et ses effets éventuels sur le contenu généré.

1.5.1 Efficacité computationnelle

- **GCT (*Grammar Compilation Time*) :** temps CPU requis pour compiler le schéma en structure interne (FSM ou CFG).

- **TTFT (*Time to First Token*) :** délai entre l'envoi de la requête et la génération du premier token.
- **TPOT (*Time per Output Token*) :** temps moyen de génération pour chaque token subséquent.



1.5.2 Couverture et conformité

On distingue trois grandeurs complémentaires, rapportées dans le benchmark sous forme normalisée :

- C_{Declared} : Couverture déclarée par le framework, c'est-à-dire la part de l'espace que le compilateur affirme pouvoir contraindre via la grammaire $L(G(\mathcal{S}))$.
- $C_{\text{Empirical}}$: Couverture observée expérimentalement, mesurée par la proportion de générations effectivement valides lors des tests.
- C_{True} : Couverture idéale, obtenue lorsque la grammaire compilée $G(\mathcal{S})$ est exactement équivalente au schéma cible $\mathcal{S}$.

1.6 Objectifs du travail

Ce projet s'inscrit dans le prolongement direct de **JSONSchemaBench**. L'objectif n'est pas uniquement de rapporter les scores obtenus par différents frameworks, mais d'interpréter ces résultats, d'en comprendre la portée et de mettre en évidence les limites techniques qu'ils révèlent.

Le travail poursuit trois objectifs principaux.

Analyser et interpréter les résultats existants.
Nous étudions les résultats présentés dans *JSONSchemaBench* afin de comprendre les écarts observés entre les frameworks de décodage contraint. L'enjeu est de dépasser les taux globaux de réussite en identifiant, de manière plus fine, les familles de schémas, les opérateurs `JSON Schema` et les étapes du pipeline susceptibles d'expliquer les échecs constatés.

Reproduire partiellement les expériences.
Nous réimplémentons une partie des pipelines expérimentaux du benchmark, dans la limite des ressources matérielles disponibles. Cette reproduction se concentre principalement sur les frameworks **Outlines** et **Guidance**, ainsi que sur une condition sans framework jouant le rôle de *baseline*. Elle vise à évaluer la reproductibilité des résultats publiés et à identifier d'éventuels écarts liés aux versions logicielles, aux choix d'implémentation ou aux contraintes d'exécution.

Compléter l'analyse par une étude des causes d'échec.
Enfin, nous prolongeons l'analyse initiale par une étude plus détaillée des erreurs observées. Nous distinguons notamment :

- les échecs de **compilation** ;
- les échecs de **génération** ;
- les échecs de **validation**.

Cette granularité permet de mieux caractériser les limites pratiques des frameworks étudiés et d'évaluer leur pertinence pour un usage réel dans des systèmes à base de LLMs.

Chapitre 2

Constrained Decoding et JSON Schema

2.1 Mécanisme du décodage sous contraintes

Pour bien saisir le fonctionnement du décodage sous contraintes, il convient de rappeler d'abord le mécanisme de génération standard d'un grand modèle de langage.

2.1.1 Génération non contrainte

Soient $\mathcal{V}$ le vocabulaire du modèle, c'est-à-dire l'ensemble des tokens susceptibles d'être produits, et $\mathcal{V}^\star$ l'ensemble de toutes les séquences de tokens de longueur arbitraire. Un LLM est un modèle autorégressif paramétré par θ qui estime la probabilité d'une séquence $y = (y_1, \ldots, y_T)$ conditionnée à une entrée x.

Remarque

$\mathcal{V}$ est fini, car le vocabulaire du modèle contient un nombre borné de tokens, tandis que $\mathcal{V}^\star$ est potentiellement infini puisqu'il regroupe des séquences de longueur arbitraire.

Par la règle de la chaîne des probabilités, la distribution jointe se décompose séquentiellement :

$$P_\theta(y \mid x) = \prod_{t=1}^{T} P_\theta(y_t \mid y_{<t}, x), \qquad \text{où } y_{<t} := (y_1, \ldots, y_{t-1}) \text{ et } y_{<1} := \varnothing. \tag{2.1}$$

À chaque étape temporelle t, le réseau produit un vecteur de scores bruts (*logits*) $l_t \in \mathbb{R}^{|\mathcal{V}|}$, puis applique la fonction Softmax :

$$P_\theta(w \mid y_{<t}, x) = \frac{\exp(l_{t,w})}{\sum_{v \in \mathcal{V}} \exp(l_{t,v})}, \qquad \forall\, w \in \mathcal{V}. \tag{2.2}$$

2.1.2 Limite du décodage standard

Dans cette formulation, le support de la distribution reste le vocabulaire entier $\mathcal{V}$. Qu'on utilise une stratégie gloutonne $y_t = \arg\max P_\theta(\cdot \mid y_{<t}, x)$ ou un échantillonnage $y_t \sim P_\theta(\cdot \mid y_{<t}, x)$, il existe un risque non nul de générer une séquence qui ne respecte pas le format attendu.

2.1.3 Formulation du décodage sous contraintes

On définit $\mathcal{C} \subseteq \mathcal{V}^\star$ comme l'ensemble des séquences valides, par exemple toutes les chaînes constituant un JSON valide au regard d'un schéma donné. On distingue alors deux stratégies de décodage. En **greedy**, on choisit la séquence la plus probable sous la contrainte :

$$y_{\text{greedy}} = \arg\max P_\theta(\cdot \mid x,\, y \in \mathcal{C}). \tag{2.3}$$

En **échantillonnage contraint**, on tire au contraire une séquence selon la loi du modèle restreinte à $\mathcal{C}$:

$$y_{\text{sample}} \sim P_\theta(\cdot \mid x,\, y \in \mathcal{C}). \tag{2.4}$$

2.1.4 Résolution locale par masquage des logits

Rechercher directement la séquence optimale dans $\mathcal{C}$ est intraitable à l'échelle globale. Le décodage sous contraintes projette donc la contrainte globale en une suite de contraintes locales strictes à chaque étape t. Pour un préfixe généré $y_{<t}$, on détermine le sous-ensemble exact des tokens qui peuvent prolonger légalement la séquence :

$$\mathcal{V}_{\text{valid}}(y_{<t}, \mathcal{C}) = \{w \in \mathcal{V} \mid \exists\, y_{\text{suffix}} \in \mathcal{V}^\star,\ (y_{<t} \oplus w \oplus y_{\text{suffix}}) \in \mathcal{C}\}. \tag{2.5}$$

On intervient alors *avant* la Softmax au moyen d'un masque additif $m_t \in \{0, -\infty\}^{|\mathcal{V}|}$:

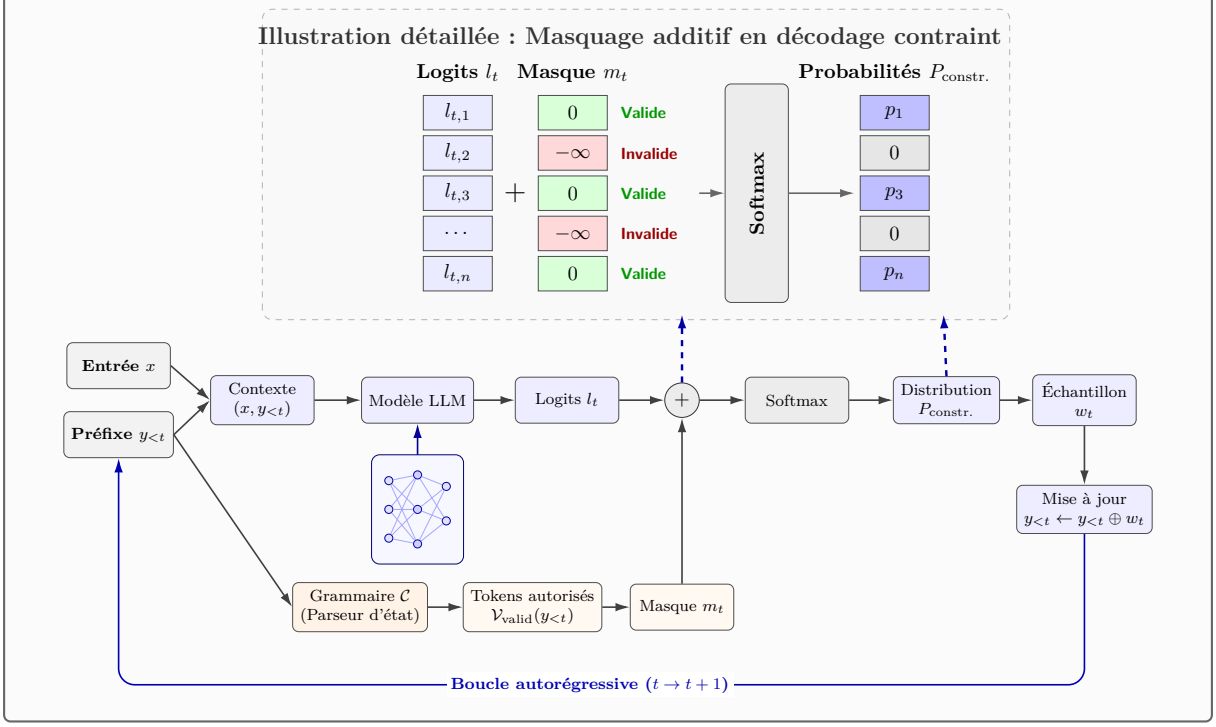
$$m_{t,w} = \begin{cases} 0, & \text{si } w \in \mathcal{V}_{\text{valid}}(y_{<t}, \mathcal{C}), \\ -\infty, & \text{sinon,} \end{cases} \tag{2.6}$$

ce qui donne la distribution effective :

$$P_{\text{constrained}}(w \mid y_{<t}, x) = \frac{\exp(l_{t,w} + m_{t,w})}{\sum_{v \in \mathcal{V}} \exp(l_{t,v} + m_{t,v})}. \tag{2.7}$$

Intuition. Si w est invalide, alors $\exp(l_{t,w} - \infty) = 0$ et sa probabilité devient exactement nulle. Si w est valide, le score relatif appris par le modèle est conservé. La renormalisation par Softmax redistribue donc toute la masse de probabilité sur les seuls tokens admissibles.

Illustration du décodage contraint



2.2 JSON Schema comme grammaire formelle

Un schéma JSON, noté $\mathcal{S}$, définit un ensemble de contraintes à la fois structurelles et sémantiques. D'un point de vue formel, $\mathcal{S}$ induit un langage $L(\mathcal{S}) \subseteq L_{\text{JSON}}$, c'est-à-dire l'ensemble des documents JSON conformes au schéma. Comme la structure du JSON est hiérarchique et potentiellement récursive, la description de $L(\mathcal{S})$ requiert en général une expressivité supérieure à celle des langages strictement réguliers. On modélise donc ce langage au moyen d'une **grammaire non contextuelle** (*Context-Free Grammar*, CFG) :

$$G = (V, \Sigma, R, S_{\text{start}}),$$

avec :

- V l'ensemble fini des symboles non terminaux ;
- Σ l'ensemble des terminaux ;
- R l'ensemble des règles de production $A \to \alpha$;
- S_{start} le symbole de départ.

L'idée centrale est qu'un schéma JSON peut être vu comme une spécification restreignant progressivement les formes syntaxiques admissibles. La grammaire associée encode alors, de manière explicite, les objets possibles, l'ordre admissible des champs, les types attendus et, dans une certaine mesure, les structures imbriquées qui peuvent apparaître dans la sortie.

2.2.1 Exemple simple : un objet utilisateur

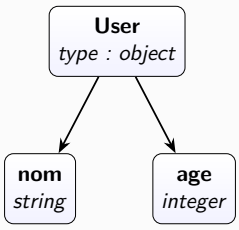
Considérons un objet utilisateur comportant deux champs obligatoires, `nom` et `age`. Le schéma JSON $\mathcal{S}_{\text{user}}$ correspondant est donné ci-dessous.

Schéma JSON $\mathcal{S}_{\text{user}}$

```
{
  "type": "object",
  "properties": {
    "nom": {"type": "string"},
    "age": {"type": "integer"}
  },
  "required": ["nom", "age"]
}
```

Le schéma peut être représenté par un arbre syntaxique abstrait simplifié, mettant en évidence la hiérarchie des champs et les types associés.

Arbre syntaxique correspondant



Ce schéma se traduit alors par une grammaire G_{user} du type suivant.

Grammaire G_{user}

$$
\begin{aligned}
S_{\text{start}} &\rightarrow \texttt{\{}\ \text{Members}\ \texttt{\}} \\
\text{Members} &\rightarrow \text{Pair}_{\text{nom}}\ \texttt{,}\ \text{Pair}_{\text{age}} \\
&\mid\ \text{Pair}_{\text{age}}\ \texttt{,}\ \text{Pair}_{\text{nom}} \\
\text{Pair}_{\text{nom}} &\rightarrow \texttt{"nom"}\ \texttt{:}\ \text{StringValue} \\
\text{Pair}_{\text{age}} &\rightarrow \texttt{"age"}\ \texttt{:}\ \text{IntValue} \\
\text{StringValue} &\rightarrow \texttt{"}\ [\texttt{\^{}"}]^{*}\ \texttt{"} \\
\text{IntValue} &\rightarrow [0\text{-}9]^{+}
\end{aligned}
$$

Explications.

- [^"] désigne un caractère différent du guillemet ".
- L'opérateur * signifie « zéro ou plusieurs répétitions ».
- L'expression [0-9] désigne un chiffre entre 0 et 9.
- L'opérateur + signifie « une ou plusieurs répétitions ».

Cette grammaire gère les deux permutations admissibles des clés obligatoires. Contrairement à une grammaire JSON générique, G_{user} n'accepte que les clés `"nom"` et `"age"`, avec les types associés.

2.2.2 Exemple composé : un objet projet

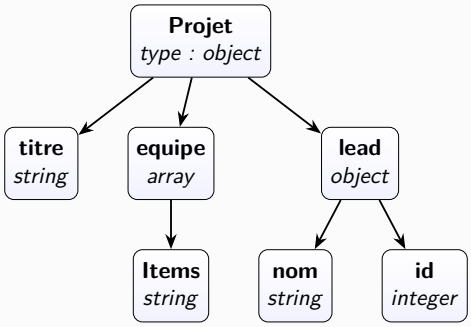
Considérons à présent un schéma plus riche, décrivant un objet `Projet`. Celui-ci contient un titre textuel, une liste de membres (*equipe*) et un sous-objet obligatoire (*lead*) possédant ses propres contraintes internes.

Schéma JSON $\mathcal{S}_{\text{projet}}$

```
{
  "type": "object",
  "properties": {
    "titre": {"type": "string"},
    "equipe": {
      "type": "array",
      "items": {"type": "string"}
    },
    "lead": {
      "type": "object",
      "properties": {
        "nom": {"type": "string"},
        "id": {"type": "integer"}
      },
      "required": ["nom", "id"]
    }
  },
  "required": ["titre", "equipe", "lead"]
}
```

L'arbre syntaxique abstrait correspondant met en évidence la présence de structures imbriquées, notamment un tableau et un sous-objet interne.

Arbre syntaxique correspondant



La grammaire associée, notée G_{projet}, est plus riche car elle doit encoder à la fois les permutations des champs obligatoires, la structure du tableau et l'objet imbriqué `lead`.

Grammaire G_{projet}

$$
\begin{aligned}
S_{\text{start}} &\rightarrow \texttt{\{}\ \text{Members}\ \texttt{\}} \\
\text{Members} &\rightarrow \text{P}_{\text{titre}}\ \texttt{,}\ \text{P}_{\text{equipe}}\ \texttt{,}\ \text{P}_{\text{lead}} \\
&\mid \text{P}_{\text{titre}}\ \texttt{,}\ \text{P}_{\text{lead}}\ \texttt{,}\ \text{P}_{\text{equipe}} \\
&\mid \text{P}_{\text{equipe}}\ \texttt{,}\ \text{P}_{\text{titre}}\ \texttt{,}\ \text{P}_{\text{lead}} \\
&\mid \text{P}_{\text{equipe}}\ \texttt{,}\ \text{P}_{\text{lead}}\ \texttt{,}\ \text{P}_{\text{titre}} \\
&\mid \text{P}_{\text{lead}}\ \texttt{,}\ \text{P}_{\text{titre}}\ \texttt{,}\ \text{P}_{\text{equipe}} \\
&\mid \text{P}_{\text{lead}}\ \texttt{,}\ \text{P}_{\text{equipe}}\ \texttt{,}\ \text{P}_{\text{titre}} \\
\text{P}_{\text{equipe}} &\rightarrow \texttt{"equipe"}\ \texttt{:}\ \texttt{[}\ \text{ArrayContent}\ \texttt{]} \\
\text{ArrayContent} &\rightarrow \text{Str} \\
&\mid \text{Str}\ \texttt{,}\ \text{ArrayContent} \\
&\mid \varepsilon \\
\text{P}_{\text{lead}} &\rightarrow \texttt{"lead"}\ \texttt{:}\ \texttt{\{}\ \text{LeadMem}\ \texttt{\}} \\
\text{LeadMem} &\rightarrow \text{P}_{\text{nom}}\ \texttt{,}\ \text{P}_{\text{id}} \\
&\mid \text{P}_{\text{id}}\ \texttt{,}\ \text{P}_{\text{nom}} \\
\text{P}_{\text{nom}} &\rightarrow \texttt{"nom"}\ \texttt{:}\ \text{Str} \\
\text{P}_{\text{id}} &\rightarrow \texttt{"id"}\ \texttt{:}\ \text{Int} \\
\text{P}_{\text{titre}} &\rightarrow \texttt{"titre"}\ \texttt{:}\ \text{Str} \\
\text{Str} &\rightarrow \texttt{"}\ [\texttt{\^{}"}]^{*}\ \texttt{"} \\
\text{Int} &\rightarrow [0\text{-}9]^{+}
\end{aligned}
$$

Explications.

- Les six premières règles de Members énumèrent les permutations admissibles des trois clés obligatoires.
- ArrayContent autorise une liste vide avec ε, un seul élément, ou plusieurs chaînes séparées par des virgules.
- LeadMem impose la présence simultanée des champs `"nom"` et `"id"`, dans n'importe quel ordre.
- [`^"`] désigne un caractère différent du guillemet `"` et $[0\text{-}9]^{+}$ une suite d'un ou plusieurs chiffres.

Ces deux exemples illustrent un point important : plus un schéma mobilise de structures imbriquées, de contraintes de cardinalité ou de permutations admissibles, plus la grammaire associée devient volumineuse. Cette croissance explique en partie pourquoi la compilation de schémas complexes peut devenir un facteur de coût significatif dans les frameworks de décodage contraint.

2.3 Écosystème des frameworks de constrained decoding

Si la formulation du masquage des logits est élégante, son application naïve est prohibitive en pratique. À chaque étape de génération, il faudrait en principe vérifier, pour chaque token du vocabulaire du modèle (souvent supérieur à 100 000 entrées) s'il peut ou non prolonger légalement le préfixe courant. Un tel test exhaustif rendrait le décodage sous contraintes trop coûteux pour un usage réel.

Pour passer à l'échelle, plusieurs frameworks ont été développés pour jouer le rôle de **compilateurs de contraintes**. Leur objectif est de traduire les règles exprimées par un schéma JSON en structures de données optimisées, interrogeables très rapidement au moment de l'inférence. Autrement dit, ces frameworks déplacent une partie de la complexité en amont, lors de la phase de compilation, afin de réduire le coût du contrôle pendant la génération.

On peut distinguer trois grandes approches aujourd'hui dominantes.

1. Outlines : approche par machine à états finis (FSM)

Outlines constitue un standard de fait dans l'écosystème Python. Le framework traduit des schémas Pydantic, des expressions régulières ou certaines formes de contraintes structurées en **machines à états finis** (FSM). Pendant la génération, la FSM maintient son état courant en fonction du préfixe produit, puis renvoie l'ensemble des tokens admissibles pour l'étape suivante.
Cette approche présente deux avantages principaux : une modélisation claire du processus de contrôle et un accès très rapide aux transitions autorisées. Toutefois, elle atteint ses limites lorsque le schéma mobilise des opérateurs très expressifs, en particulier certains combinateurs logiques comme `oneOf` ou `not`. Dans ces cas, la représentation par FSM peut devenir très coûteuse, voire inadaptée.

2. Guidance : génération entrelacée et parties forcées

Développé par Microsoft, **Guidance** adopte une stratégie différente. Plutôt que de raisonner uniquement en termes de filtrage des logits sur tout le vocabulaire, il distingue les segments fixes du format des segments réellement variables. Les fragments déterministes du schéma peuvent alors être injectés directement dans le *KV-cache* sous la forme d'un *prefill* forcé, tandis que la génération autorégressive est réservée aux seules parties ouvertes.
Cette stratégie peut réduire le nombre de *forward passes* nécessaires, notamment lorsque la structure de sortie est très verbeuse. En contrepartie, le modèle d'exécution est moins naturellement formulé comme un simple masquage global des logits, puisqu'il repose sur une alternance entre écriture forcée et génération libre sous contrôle.

3. XGrammar : compilation bas niveau orientée GPU

Développé dans l'écosystème MLC, pousse plus loin l'optimisation de bas niveau. Le framework compile les contraintes dans des structures adaptées à une exécution proche du moteur d'inférence, avec une attention particulière portée au coût du masquage sur GPU. En pratique, il pré-alloue des *bitmasks* en mémoire vidéo et applique le filtrage au moyen d'opérations très efficaces, voire de noyaux CUDA dédiés.
L'intérêt de cette approche réside dans la réduction du surcoût d'inférence, même lorsque le vocabulaire est très large. En revanche, cette efficacité suppose une intégration plus étroite avec les couches basses du système, ce qui rend souvent l'approche moins flexible que les

solutions purement Python.

La différence entre ces frameworks ne porte donc pas uniquement sur leur interface de programmation. Elle concerne surtout le **niveau auquel la contrainte est compilée et appliquée** : représentation symbolique explicite, interleaving avec *prefill*, ou masquage natif proche du moteur GPU.

Architecture en couches du décodage contraint

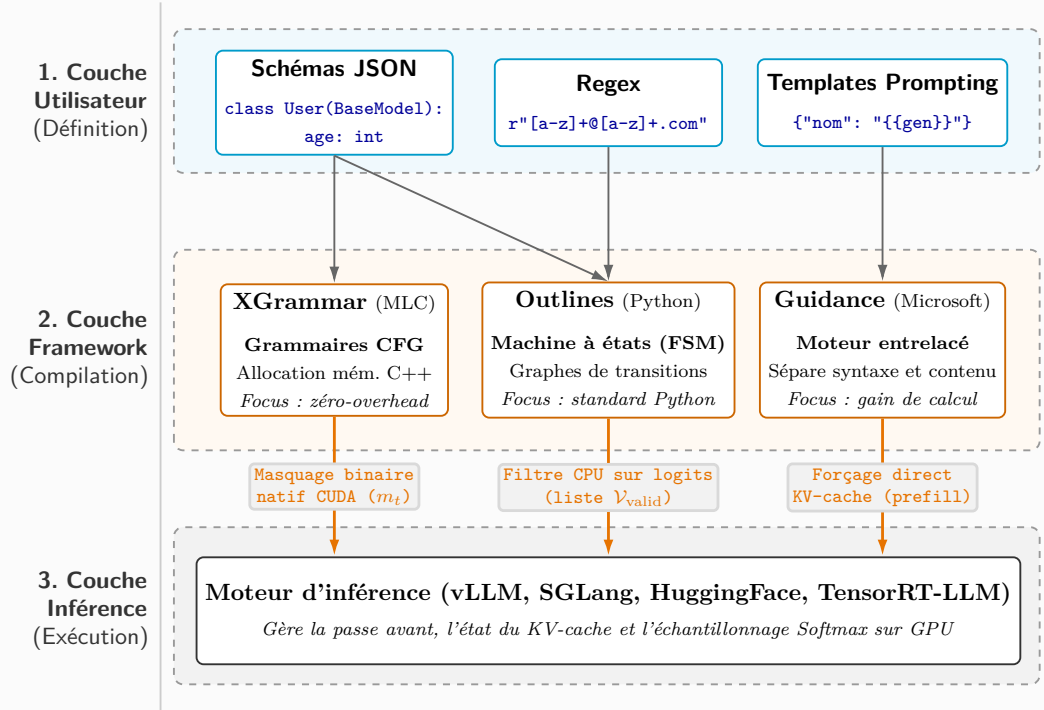
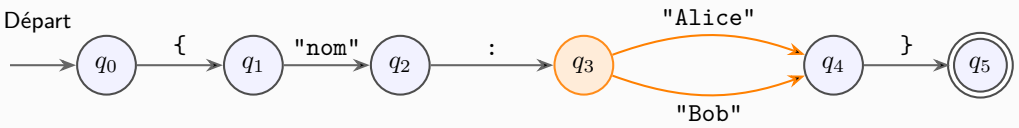


Illustration : masquage des logits par machine à états finis



Lorsque l'automate est dans l'état courant q_3, seules deux continuations sont autorisées : `"Alice"` et `"Bob"`. Le framework n'a donc pas besoin d'examiner l'intégralité du vocabulaire ; il lui suffit de consulter la table d'index associée à cet état, puis de transformer ce résultat en masque sur les logits.

Index en mémoire

État	Index en $O(1)$
q_0	`{`
q_1	`"nom"`
q_2	`:`
q_3	`"Alice", "Bob"`
q_4	`}`

Masque appliqué au vocabulaire

Token	Masque
`{`	$-\infty$
`"nom"`	$-\infty$
`:`	$-\infty$
`"Alice"`	**autorisé**
`"Bob"`	**autorisé**
`}`	$-\infty$
…	$-\infty$

Chapitre 3

Contributions expérimentales

3.1 Environnement d'expérimentation et ressources

L'ensemble de nos expérimentations a été conduit sur les infrastructures GPU mises à disposition par la **PPTI** (Plateforme Pédagogique de Sorbonne Université). Cet environnement a permis d'exécuter les pipelines de reproduction du *JSONSchemaBench* via *SSH*, mais il a également imposé plusieurs contraintes pratiques qui doivent être prises en compte dans l'interprétation de nos résultats.

Nos expériences ont principalement été réalisées sur le serveur `ppti-gpu-4`.

Illustration : Configuration matérielle et logicielle de notre environnement d'expérimentation.

Composant	**Configuration**
GPU	4 GPU Tesla V100-SXM2
VRAM	32 Go
RAM	32 Go
Framework Outlines	v0.1.8 + `outlines-core`
Framework Guidance	v0.3 + `llguidance`
Modèle LLM	Llama-3.1-1B-Instruct

Contraintes identifiées. Nos expérimentations ont été limitées par plusieurs contraintes pratiques liées à l'utilisation d'une plateforme GPU mutualisée.

Partage des ressources. La première difficulté vient du fait que les machines de la PPTI sont partagées entre plusieurs utilisateurs. Même si le serveur utilisé dispose théoriquement de plusieurs GPU, les ressources réellement disponibles variaient fortement. En pratique, nous avons souvent travaillé avec une fraction seulement de la VRAM, parfois autour de **30 % de mémoire libre**. Cette contrainte a eu un impact direct sur l'exécution des modèles, en particulier lors du traitement de schémas volumineux.

Stabilité des exécutions. La deuxième difficulté concerne la stabilité des exécutions longues. Les scripts pouvaient s'interrompre pour diverses raisons : dépassement mémoire (*Out Of Memory*), *timeout* ou crash du processus Python. Nous avons donc dû mettre en place des mécanismes de relance, notamment sous la forme d'un **watchdog**, afin de reprendre automatiquement les expériences et d'éviter la perte de données.

Environnement logiciel. L'installation d'un environnement compatible avec les versions nécessaires des modèles et des frameworks (*Outlines*, *Guidance*) se sont révélées coûteuses en temps. Ces difficultés sont classiques en reproduction : les résultats dépendent fortement des versions exactes des bibliothèques et de leurs dépendances internes.

Stockage et modèles. Nous avons également été limités par l'espace de stockage, notamment pour la gestion des caches, des modèles et des fichiers CSV de résultats. Enfin, pour des raisons de stabilité, nous avons privilégié un modèle de taille **1B** plutôt qu'un modèle **8B**.

Ce choix introduit une limite importante : le modèle 1B possède une fenêtre de contexte plus restreinte. Pour les schémas JSON volumineux, le prompt complet (exemples, instructions et schéma cible) pouvait dépasser la longueur d'entrée acceptable, provoquant des échecs spécifiques liés à la saturation du contexte.

Ces contraintes expliquent le périmètre retenu : nous avons privilégié l'analyse des **causes d'échec** plutôt qu'un benchmark exhaustif de performance temporelle. Nos résultats doivent donc être interprétés comme une reproduction partielle sous contraintes matérielles.

3.2 Évaluation empirique : le *JSONSchemaBench* et ses pipelines

Pour évaluer objectivement les métriques d'efficacité et de couverture des frameworks de décodage contraint, il est nécessaire de s'appuyer sur un corpus de test vaste, diversifié et représentatif.

3.2.1 La base de données *JSONSchemaBench*

Le *JSONSchemaBench* a été conçu pour répondre à ce besoin. Il rassemble près de 10 000 schémas issus de la réalité industrielle et open-source. La collecte s'articule autour de plusieurs sources majeures :

- **GitHub (Baazizi et al., 2021) :** un ensemble de 6 000 schémas extraits de dépôts publics, présentant une grande variance de complexité ;
- **Standards et API :** des schémas issus de la *JSON Schema Test Suite*, du *JSON Schema Store*, et de fichiers *Kubernetes* ;
- **Appels de fonctions :** le jeu GlaiveAI V2, pertinent pour évaluer la capacité des LLMs à utiliser des outils externes.

Jeu de données	**Catégorie (usage)**	**Nombre de schémas**
GlaiveAI-2K	Appels de fonctions (function call)	1 707
Github-Trivial	Divers (*exclu pour évaluation standard*)	444
Github-Easy	Divers	1 943
Snowplow	API opérationnelle	403
Github-Medium	Divers	1 976
Kubernetes	API Kubernetes	1 064
WashingtonPost	API d'accès aux ressources	125
Github-Hard	Divers	1 240
JSONSchemaStore	Divers	492
Github-Ultra	Divers (*cible pour la recherche future*)	164
Total		**9 558**

TABLE 3.1 – Métadonnées de la collection du *JSONSchemaBench*.

Ce corpus couvre ainsi plusieurs niveaux de difficulté. Il combine des schémas simples, proches de cas d'usage pédagogiques, avec des schémas beaucoup plus riches mobilisant des objets imbriqués, des contraintes combinatoires et des structures proches de situations industrielles réelles.

3.2.2 Pipeline d'évaluation de la couverture

L'objectif de ce pipeline est de vérifier empiriquement que le framework respecte effectivement les contraintes exigées par le schéma. Pour isoler les erreurs inhérentes au framework (par exemple les fuites du compilateur) de celles liées au modèle de langage lui-même, l'évaluation de la couverture est séparée en deux sous-pipelines complémentaires. Nous présentons ici le premier, correspondant à une évaluation avec LLM.

Pipeline A : Validation d'instance sur JSONSchemaBench . Ce test vérifie le comportement global du système complet (framework, compilation des contraintes et modèle de langage) face à un schéma complet.

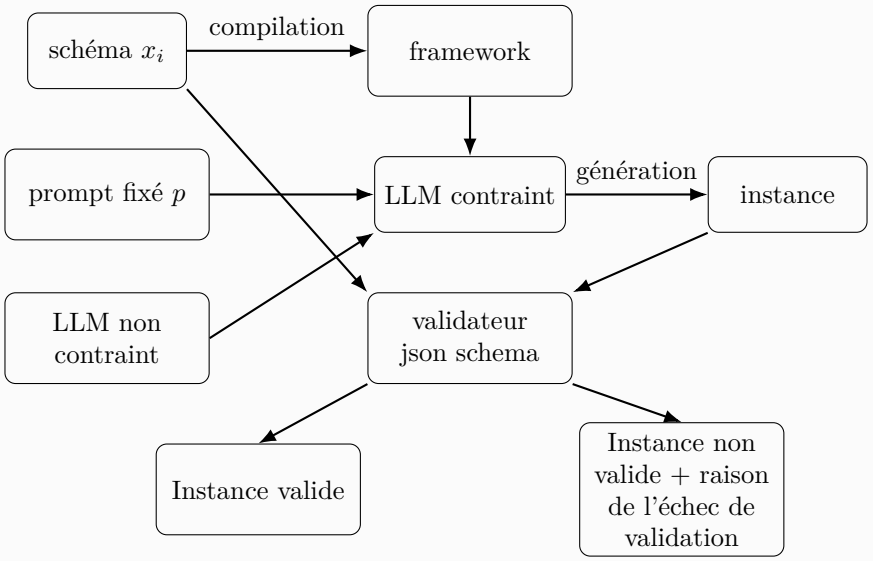
1. **Extraction depuis la base :** les schémas sont parcourus depuis la base *JSONSchemaBench*.
2. **Génération (framework + LLM) :** pour chaque schéma, le framework compile les contraintes, puis le LLM génère une instance textuelle sous contrainte.
3. **Validation externe :** chaque échantillon généré est exporté puis évalué par un validateur JSON externe de référence, par exemple `jsonschema`. Le validateur agit comme une fonction indicatrice renvoyant 1 si l'instance est valide relativement au schéma correspondant, et 0 sinon.

Métriques et signaux collectés.

- **Empirical rate :** proportion totale d'instances valides générées sur l'ensemble du jeu de données.
- **Temps de génération :** temps requis par le couple (framework, LLM) pour produire les instances.
- **Erreur de compilation :** cas où le framework refuse de démarrer la génération par exemple en raison d'une clause non couverte ou d'un schéma trop volumineux.
- **Erreur de génération :** cas où le framework accepte de compiler et d'initialiser la génération, mais ne fournit pas de sortie complète, par exemple à cause d'un *timeout* ou d'un échec technique au moment de l'inférence.

- **Erreur de validation :** cas où le framework compile et la génération aboutit à une sortie complète, mais cette sortie échoue à la validation des contraintes.

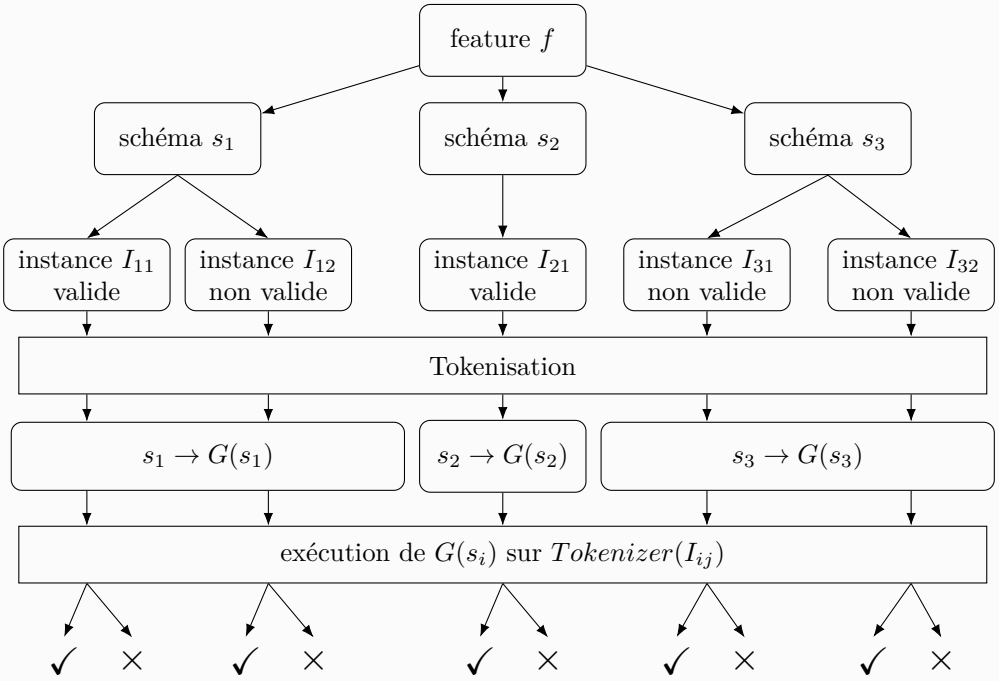
Schéma de validation d'une génération contrainte



Pipeline B : JSON Schema Test Suite sans LLM. Certains frameworks peuvent paraître corrects lorsqu'ils sont utilisés avec un LLM, alors qu'ils restent en réalité trop permissifs ou trop restrictifs. Le pipeline illustré par la Figure 3.2.2 élimine ce biais en testant directement le comportement du framework sur des instances contrôlées.

1. **Regroupement par feature :** on part d'une feature JSON Schema cible f et on a un ensemble de schémas s_i qui mobilisent ce mot-clé ou cette construction, par exemple `maxLength`, `pattern`, `anyOf` ou `enum`.
2. **Instances valides/ non valides :** pour chaque schéma s_i, on a plusieurs instances I_{ij}, certaines **valides** et d'autres **invalides**. Donc un même schéma peut être relié à plusieurs instances et on distingue séparément les cas attendus et les contre-exemples.
3. **Tokenisation :** chaque instance est sérialisée puis tokenisée avec le tokeniseur du framework évalué. On obtient ainsi exactement la séquence de tokens que le système devrait accepter ou rejeter.
4. **Compilation du schéma + Execution ($Tokenizer(s_i) \overset{?}{\in} L(G(s_i))$) :** chaque schéma s_i est compilé dans le formalisme interne du framework, noté ici $G(s_i)$, puis cette grammaire est exécutée sur chaque séquence `Tokenizer(`I_{ij}`)` correspondante. Le point important est qu'on conserve la multiplicité visible dans le schéma : un même schéma compilé est testé sur toutes les instances qui lui sont associées.
5. **Interprétation des verdicts :** si le framework rejette trop d'instances valides, on le considère comme *overconstrained*. À l'inverse, s'il accepte trop d'instances invalides, on le considère comme *underconstrained*. Enfin, dans le cas particulier d'un schéma ne comportant que des instances invalides, si le framework ne parvient pas à le compiler, on considère ce schéma comme **réussi**, car l'échec de compilation empêche précisément l'acceptation de contre-exemples qui auraient dû être refusés.

Arbre de schémas, instances et exécution des grammaires



3.3 Expérimentations et Résultats

Cette section présente notre campagne d'évaluation empirique. Conformément à nos capacités de calcul, nous avons restreint le périmètre d'analyse à deux frameworks majeurs : **Outlines** (approche par machine à états) et **Guidance** (approche dynamique par fast-forwarding).

3.3.1 Résultat de l'article (JSONSchemaBench)

Avant de présenter nos propres résultats expérimentaux, nous revenons d'abord sur les principaux résultats obtenus dans l'article introduisant le JSONSchemaBench (Geng et al., 2025), afin de comprendre les comportements observés des différents frameworks de décodage contraint. Ce papier de recherche évalue six moteurs de décodage sous contraintes (Guidance, Outlines, Llamacpp, XGrammar, ainsi que les API d'OpenAI et Gemini) selon trois axes : l'efficacité temporelle, la couverture des schémas et la qualité sémantique.

Les auteurs démontrent que la méthode employée pour contraindre le modèle a un impact radical sur les performances. Leurs conclusions se divisent en trois points :

- **Efficacité computationnelle :**
  En théorie, calculer un masque de validité à chaque token devrait ralentir le modèle. Pourtant, l'étude montre que le décodage sous contraintes peut être jusqu'à **50 % plus rapide** qu'une génération libre.
  - **Guidance est le plus rapide**, car il utilise une technique de *fast-forwarding* (génération accélérée). Au lieu de demander au LLM de deviner et générer les caractères de structure (comme `{"` ou `:[`), Guidance les injecte directement dans le cache du LLM. Le LLM ne calcule alors que le contenu des variables, ce qui réduit fortement le coût de génération.
  - **Outlines est le plus lent au démarrage**. Son approche consiste à compiler le schéma JSON en une grande expression régulière ou en machine à états finis. Sur des schémas contenant des tableaux (`array`) ou des bornes (`maxItems`), cette conversion provoque parfois une explosion combinatoire. Le temps de compilation (GCT) peut alors bloquer l'exécution pendant plusieurs dizaines de secondes, voire plusieurs minutes, avant même que le premier token ne soit généré.
- **Couverture et fiabilité :**
  L'étude a testé la capacité des frameworks à gérer les différentes règles du standard JSON Schema. Les auteurs identifient deux types de défaillances opposées :
  - **Les moteurs sur-contraints (Outlines, Llamacpp) :** face à une règle JSON complexe, ils peuvent soit rejeter un token pourtant valide, soit échouer dès la compilation lorsqu'une règle n'est pas implémentée en interne.
  - **Les moteurs sous-contraints (XGrammar) :** pour aller plus vite, ils ignorent certaines contraintes complexes du schéma. Ils deviennent alors trop permissifs et laissent le LLM générer des erreurs, en comptant sur le modèle pour ne pas se tromper.
  - **Le meilleur compromis (Guidance) :** il offre la meilleure couverture empirique globale, car il équilibre le respect strict des contraintes avec une architecture qui évite la plupart des explosions de complexité observées sur les approches purement automatiques.

En résumé, l'état de l'art désigne **Guidance** comme la solution la plus robuste et la plus rapide grâce à son ingénierie d'injection directe, tandis qu'**Outlines**, bien que élégant, se heurte plus rapidement aux limites physiques de la compilation de graphes complexes. Ce sont ces dynamiques précises que nous analysons dans la suite de nos expériences.

3.3.2 Notre campagne d'expérimentation

Pour comprendre, reproduire et étendre les observations de la littérature, nous avons implémenté les deux pipelines d'évaluation décrits précédemment. L'objectif est double : isoler le comportement formel des compilateurs (Pipeline B) et mesurer la robustesse en conditions réelles avec un LLM (Pipeline A).

Résultats de la Test Suite (Pipeline B : évaluation par feature)

Dans cette première expérience, le LLM est désactivé. L'objectif est d'évaluer de manière purement déterministe si les compilateurs génèrent des structures internes strictement équivalentes aux règles du standard JSON Schema.

Plutôt que de nous contenter des résultats publiés dans le *JSONSchemaBench*, nous avons mené une **rétro-ingénierie complète de leur pipeline d'évaluation** afin de vérifier la reproductibilité des scores. Nos travaux révèlent plusieurs nuances méthodologiques qui ne sont pas toujours explicitées dans l'article.

TABLE 3.2: **Analyse de la couverture déterministe (JSON Schema Test Suite).** Comparaison entre nos reproductions strictes et les scores du papier original. Les flèches soulignent les écarts de performance.

Feature JSON	Base.	Outlines			Guidance		
		Notre	Papier	Δ	Notre	Papier	Δ
`add.Properties`	0.22	0.22	0.22	=	0.78	0.67	**+0.11** ↑
`allOf`	0.17	0.42	0.42	=	0.75	0.75	=
`anchor`	0.00	0.75	0.00	**+0.75** ↑	1.00	1.00	=
`anyOf`	0.13	0.38	0.38	=	1.00	1.00	=
`boolean_schema`	0.50	0.50	0.50	=	1.00	1.00	=
`const`	0.00	0.47	0.33	**+0.14** ↑	0.59	0.60	**-0.01** ↓
`contains`	0.00	0.00	0.00	=	0.00	0.00	=
`content`	0.00	0.00	0.00	=	1.00	1.00	=
`default`	0.00	0.67	0.67	=	1.00	1.00	=
`defs`	0.00	0.00	0.00	=	0.00	0.00	=
`dependentReq.`	0.00	0.00	0.00	=	0.00	0.00	=
`dependentSch.`	0.00	0.00	0.00	=	0.00	0.00	=
`dynamicRef`	0.00	0.00	0.00	=	0.00	0.00	=
`enum`	0.07	0.53	0.50	**+0.03** ↑	0.73	0.64	**+0.09** ↑
`exclusiveMax.`	0.00	0.00	0.00	=	1.00	1.00	=
`exclusiveMin.`	0.00	0.00	0.00	=	0.00	1.00	**-1.00** ↓
`if-then-else`	0.00	0.00	0.00	=	0.00	0.00	=
`infinite-loop`	0.00	0.00	0.00	=	1.00	1.00	=
`items`	0.00	0.10	0.10	=	1.00	1.00	=
`maxContains`	0.00	0.00	0.00	=	0.00	0.00	=
`maxItems`	0.00	0.00	0.00	=	0.50	0.50	=
`maxLength`	0.00	0.00	0.00	=	0.00	0.00	=
`maxProperties`	0.00	0.00	0.00	=	0.67	0.00	**+0.67** ↑
`maximum`	0.00	0.00	0.00	=	1.00	1.00	=

Suite à la page suivante...

Table 3.2 – Suite de la page précédente

Feature JSON	Base.	Outlines			Guidance		
		Exp	Papier	Δ	Exp	Papier	Δ
`minContains`	0.13	0.13	0.12	**+0.01** ↑	0.13	0.12	**+0.01** ↑
`minItems`	0.00	0.00	0.00	=	0.50	0.50	=
`minLength`	0.00	0.00	0.00	=	0.50	0.50	=
`minProperties`	0.00	0.00	0.00	=	0.50	0.00	**+0.50** ↑
`minimum`	0.00	0.00	0.00	=	1.00	1.00	=
`multipleOf`	0.20	0.40	0.40	=	0.60	0.20	**+0.40** ↑
`not`	0.22	0.22	0.22	=	0.22	0.22	=
`oneOf`	0.27	0.46	0.45	**+0.01** ↑	0.46	0.27	**+0.19** ↑
`pattern`	0.00	0.00	0.00	=	0.67	1.00	**-0.33** ↓
`patternProps`	0.00	0.00	0.00	=	0.50	0.00	**+0.50** ↑
`prefixItems`	0.00	0.25	0.25	=	1.00	1.00	=
`properties`	0.00	0.17	0.17	=	0.83	0.83	=
`propertyNames`	0.00	0.00	0.00	=	0.00	0.00	=
`ref`	0.06	0.33	0.34	**-0.01** ↓	0.89	0.86	**+0.03** ↑
`required`	0.00	0.40	0.40	=	1.00	0.80	**+0.20** ↑
`type`	0.00	0.64	0.55	**+0.09** ↑	0.91	0.91	=
`unevaluatedIt.`	0.10	0.17	0.15	**+0.02** ↑	0.10	0.07	**+0.03** ↑
`unevaluatedPr.`	0.14	0.21	0.25	**-0.04** ↓	0.14	0.12	**+0.02** ↑
`uniqueItems`	0.00	0.17	0.17	=	0.00	0.00	=

Analyse du tableau Ce tableau met en évidence trois phénomènes distincts :

1. **Le comportement conservateur d'Outlines.** Le framework obtient des scores corrects sur des fonctionnalités simples ou localement vérifiables, telles que `type`, `enum`, `default` ou `prefixItems`. En revanche, il échoue rapidement sur des contraintes plus globales ou logiques, comme `dependentRequired`, `dependentSchemas`, `if-then-else`, `contains` ou `uniqueItems`. Cela confirme que son approche par expressions régulières et machines à états est efficace sur un sous-ensemble bien délimité de `JSON Schema`, mais moins adaptée aux opérateurs nécessitant un raisonnement global sur l'instance.
2. **La robustesse relative de Guidance.** Globalement plus robuste, il atteint de bons scores sur davantage de fonctionnalités, notamment `anyOf`, `items`, `properties`, `required` ou les bornes numériques. En revanche, les écarts observés par rapport aux résultats de l'article original montrent que cette robustesse dépend fortement de la version de la bibliothèque et du moteur sous-jacent.
3. **Les limites méthodologiques de la baseline.** La colonne de *baseline* révèle un point important du protocole de *Test Suite*. On pourrait s'attendre à ce qu'une stratégie triviale consistant à rejeter systématiquement toutes les instances obtienne un score nul. Or, certaines fonctionnalités affichent une *baseline* non nulle, indiquant que certains groupes de tests ne contiennent aucune instance valide.

Cette observation impose une lecture prudente des résultats : un score positif ne prouve pas nécessairement qu'un framework supporte réellement une fonctionnalité, puisqu'il peut parfois refléter un simple comportement de rejet systématique.

Résultats du JSON Schema Bench (Pipeline A : évaluation globale)

Cette phase évalue le système complet framework plus LLM sur l'ensemble des 9 558 schémas du benchmark. Contrairement à l'article qui présente ses résultats sous dataset par sous dataset, nous avons choisi une analyse globale centrée sur les causes d'échec à chaque étape du pipeline : compilation, génération et validation. Cette granularité, absente du papier original, constitue notre principale contribution sur ce pipeline. Nous présentons successivement les trois conditions expérimentales : le modèle sans contrainte comme baseline, puis Outlines, puis Guidance.

1. Comportement de base (LLM Only) Afin d'établir une baseline, nous avons d'abord évalué le modèle de langage sans aucun framework de décodage sous contraintes (LLM Only), guidé uniquement par le prompt classique avec contrainte.

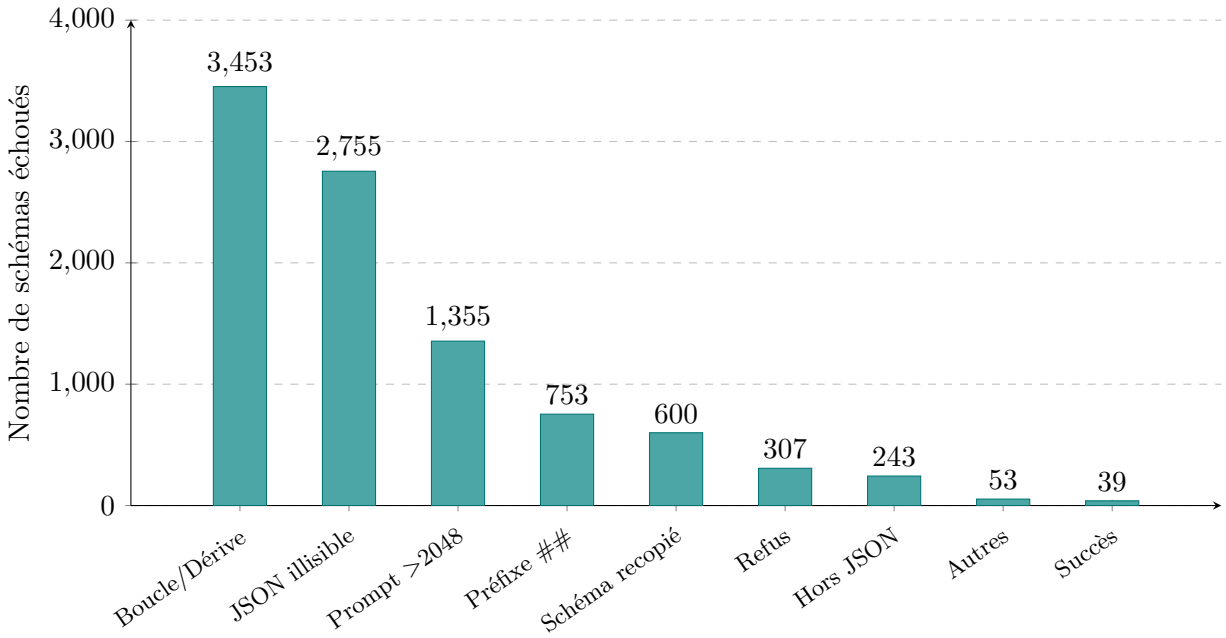


FIGURE 3.1 – **Causes d'échec ou réussite du modèle Only**.

Cette *baseline* montre que le LLM seul est incapable de garantir une sortie structurée fiable : sur **9 558 schémas**, seuls **39 succès** sont obtenus, soit environ **0,4 %**. Ces rares réussites correspondent vraisemblablement à des cas triviaux où le modèle produit par chance une instance valide.

La majorité des échecs provient de dérives de génération diverses :

- **Dérives textuelles :** le modèle continue le prompt, répète le schéma au lieu de l'instancier, ou ajoute du texte naturel autour du bloc JSON.
- **Erreurs de formatage :** présence de préfixes parasites (comme `##`) ou production d'un JSON superficiellement correct mais syntaxiquement illisible.
- **Contraintes techniques :** des prompts trop longs, liés aux limites de la fenêtre de contexte du modèle **1B** face aux schémas volumineux, ainsi que quelques refus explicites de génération.

Ces résultats confirment que le *prompting* seul ne suffit pas : sans contrainte appliquée directement pendant le décodage, le modèle peut à tout moment dévier du format attendu, même lorsque la consigne impose explicitement une instance `JSON` conforme.

2. Outlines : le goulot d'étranglement de la compilation FSM Outlines base son architecture sur la traduction du schéma JSON en expressions régulières puis en machines à états finis (FSM). L'analyse de ses défaillances révèle que cette technique se heurte à des limites.

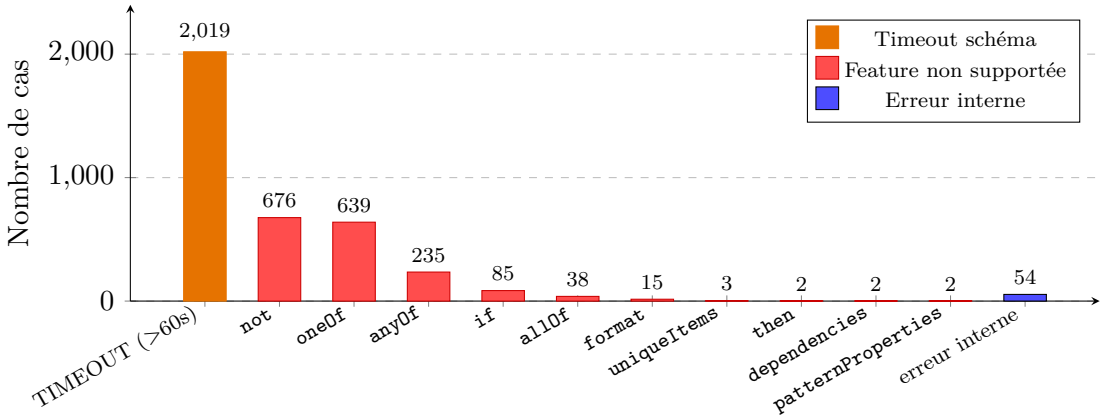


FIGURE 3.2 – **Causes d'échec de compilation d'Outlines.**

Échecs de compilation. L'analyse des échecs de compilation montre que la principale faiblesse d'*Outlines* se situe en amont même de la génération. La cause dominante est de loin le **timeout de compilation (2 019 cas)**, indiquant que de nombreux schémas deviennent trop coûteux à traduire dans la représentation interne du framework.

Les opérateurs logiques et compositionnels posent également problème :

- `not` : 676 cas ;
- `oneOf` : 639 cas ;
- `anyOf` : 235 cas ;
- `if`, `allOf` et `format` : impact plus marginal (respectivement 85, 38 et 15 cas).

Ce profil confirme qu'*Outlines* gère relativement bien les contraintes locales, mais se heurte aux structures difficiles à encoder sous forme de *regex* ou de machine à états. Enfin, les **54 erreurs internes** témoignent d'une certaine fragilité de l'implémentation sur certains schémas complexes.

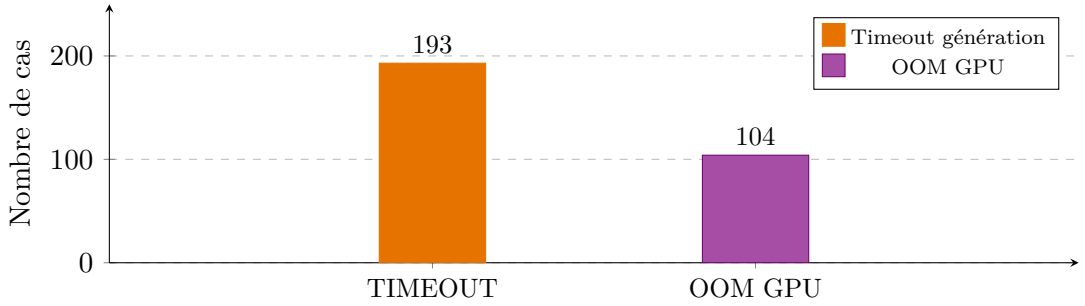


FIGURE 3.3 – **Cause d'échecs de génération d'Outlines.**

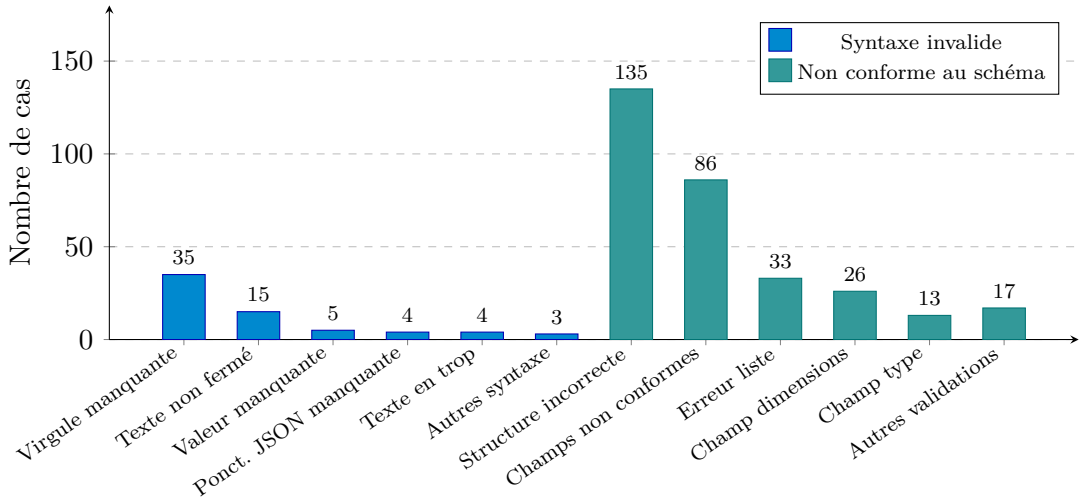


FIGURE 3.4 – **Analyse d'échecs de validation d'Outlines.**

Échecs de génération et de validation. Même lorsque la compilation et la génération aboutissent, une étape de validation reste nécessaire. Bien que minoritaires, ces erreurs montrent qu'*Outlines* ne garantit pas systématiquement une sortie correcte de bout en bout :

Syntaxe invalide : correspond à des fichiers `JSON` impossibles à parser (virgule manquante, guillemets non fermés, etc.).

Non-conformité au schéma : cas le plus fréquent où le `JSON` est lisible mais invalide au regard du contrat. On dénombre notamment :

- **structures incorrectes** (135 cas) ;
- **champs non conformes** (86 cas) ;
- des erreurs sur les listes, les dimensions ou les types de champs.

En résumé, la validité syntaxique ne suffit pas : une sortie peut être bien formée tout en restant invalide au regard du `JSON Schema` cible.

Enfin, les **échecs de validation** (Figure 3.4) révèlent un paradoxe pour un framework de décodage guidé. On note des sorties sémantiquement ou syntaxiquement invalides, avec notamment 135 cas de structures globales incorrectes et 35 erreurs basiques de ponctuation. Ces défaillances suggèrent des limites dans la précision du masquage des logits ou des comportements de *fallback* imprévus lorsque la FSM rencontre des ambiguïtés lors de la projection du masque.

3. Guidance : Guidance adopte une approche radicalement différente, fondée sur une approche dynamique par fast-forwarding.

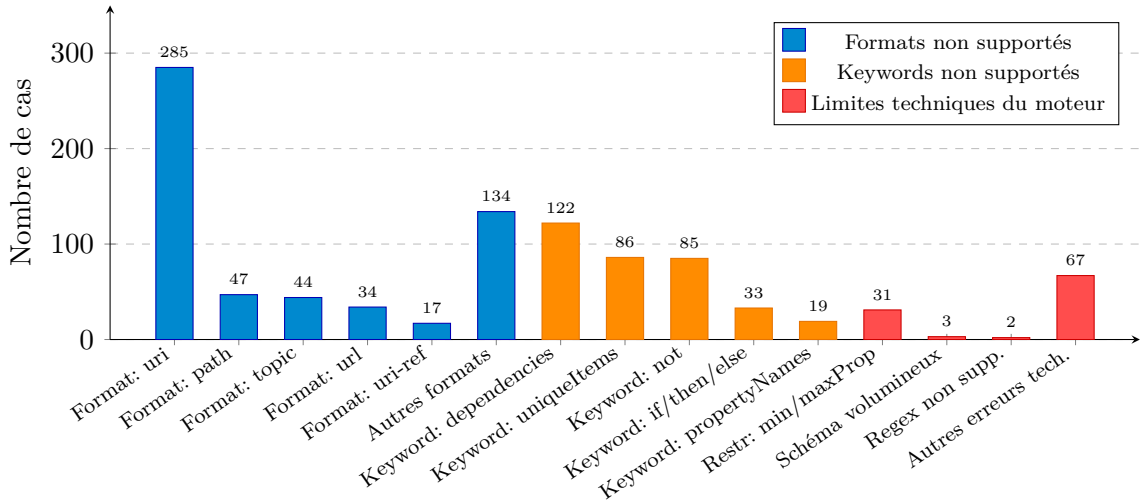


FIGURE 3.5 – **Causes d'échecs de compilation pour Guidance.**

Pour *Guidance*, les échecs sont moins variés que pour *Outlines* et se concentrent autour de deux phénomènes principaux.

Échecs de compilation. Ces échecs proviennent principalement de contraintes explicitement non supportées par le moteur :

- **formats spécifiques :** `uri`, `relative-json-pointer`, `json-pointer`, etc. ;
- **mots-clés :** `dependencies`, `uniqueItems`, `not`, `if/then/else` ou `propertyNames`.

Cela montre que *Guidance* adopte une **stratégie de prudence** : lorsqu'une contrainte n'est pas prise en charge, le framework préfère la refuser plutôt que de produire une grammaire approximative. Quelques limites techniques (schémas trop longs, erreurs internes) subsistent mais restent marginales.

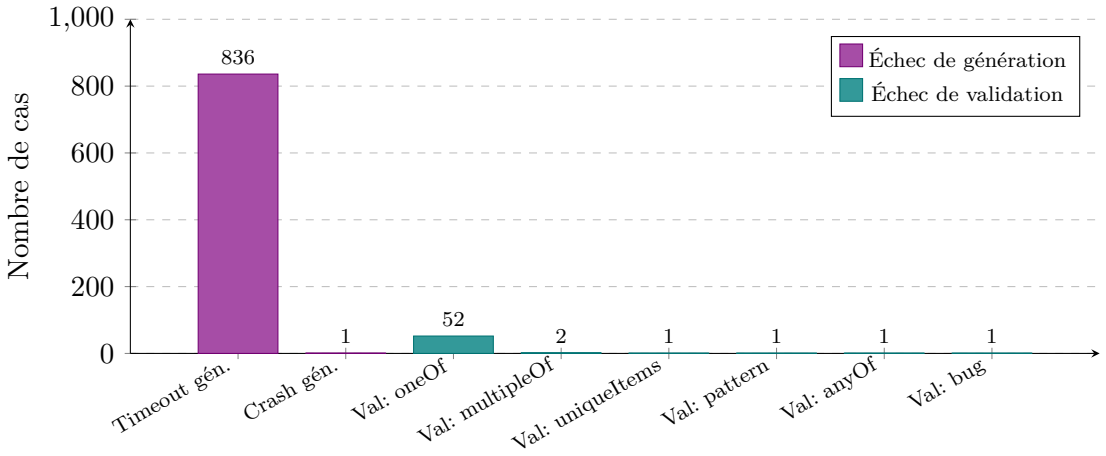


FIGURE 3.6 – **Causes d'échecs de génération et de validation pour Guidance.**

Phase de génération et de validation. Les résultats confirment la robustesse de ce framework, tout en soulignant un coût computationnel élevé :

- **timeout de génération :** c'est le problème majeur avec **836 cas**, dépassant de loin toutes les autres erreurs ;
- **erreurs de validation :** elles sont très rares. On observe principalement des échecs liés au `oneOf` (52 cas), puis quelques cas isolés sur `multipleOf`, `uniqueItems`, `pattern` ou `anyOf`.

Conclusion sur Guidance. Sa faiblesse principale n'est pas la production de `JSON` invalide, mais plutôt le **refus préventif** de certaines fonctionnalités et le **coût temporel** de génération sur des schémas complexes. Lorsque le framework accepte un schéma et termine le processus, la sortie est généralement conforme.

3.4 Analyse comparative et apports

3.4.1 Nécessité d'un framework de *Constrained Decoding*

Le bond de performance entre le LLM libre et les frameworks contraints est radical. Sans framework, le modèle ne produit que 39 sorties JSON utilisables sur plusieurs milliers de tentatives (taux de succès inférieur à 1 %). Avec un framework, même imparfait, les erreurs syntaxiques basiques sont éliminées par construction. L'utilisation d'un framework de décodage contraint n'est donc pas une simple optimisation : c'est une **condition nécessaire** à tout déploiement fiable en production.

3.4.2 Synthèse comparative : Outlines vs Guidance

Framework	**Taux succès**	**Succès/post compil**	**Erreurs/post-gen**
Outlines	54%	5160/5827 (88.6%)	371/5531 (6.7%)
Guidance	80%	7654/8549 (89.5%)	59/7713 (0.8%)

Ce tableau révèle un résultat contre-intuitif. Sur le taux de succès post-compilation, *Outlines* et *Guidance* sont quasiment équivalents : 88,6 % contre 89,5 %.

La différence principale ne réside donc pas dans la fiabilité une fois la compilation réussie, mais dans la couverture à la compilation. *Guidance* accepte 8 549 schémas sur 9 558, là où *Outlines* n'en accepte que 5 827. C'est ce différentiel de couverture (et non une meilleure qualité intrinsèque de génération) qui explique l'écart de taux de succès global entre les deux frameworks (80,1 % contre 54,0 %).

En revanche, la différence devient nette concernant la conformité des *outputs* rendus. Lorsque le framework produit effectivement un résultat, *Outlines* génère 6,7 % de sorties invalides contre seulement 0,8 % pour *Guidance*. Ce dernier garantit donc mieux la conformité finale sur les schémas qu'il accepte.

Ces chiffres doivent toutefois être interprétés avec précaution. Notre pipeline impose un *timeout* de 60 secondes à la compilation, ce qui pénalise structurellement *Outlines*. En production, les grammaires sont généralement compilées une seule fois en amont puis réutilisées, ce qui effacerait en grande partie ce désavantage.

Nos résultats reflètent donc autant nos choix d'implémentation que les capacités intrinsèques des frameworks.

Outlines : échecs précoces et limites structurelles. *Outlines* repose sur une traduction du *JSON Schema* vers des machines à états finis (*Finite State Machines*, FSM). Cette approche

est efficace pour les contraintes locales (types simples, `enum`, structures régulières) mais devient fragile dès que le schéma contient des opérateurs logiques tels que `not`, `oneOf`, `anyOf` ou `if`.

Les échecs surviennent majoritairement à la compilation, souvent avant même que le LLM ait produit un seul token. Ces limites sont structurelles : une FSM ne peut pas encoder naturellement ce type de constructions.

Guidance : échecs tardifs et conformité garantie. *Guidance* adopte une stratégie différente. Le framework rejette précocement les schémas qu'il ne peut pas gérer (formats non supportés, *keywords* complexes) mais garantit ensuite la conformité sur les schémas acceptés.

Les erreurs restantes se concentrent principalement dans la phase de génération, sous la forme de *timeouts*. Ceux-ci sont une conséquence directe du *template interleaving*, qui contraint la génération token par token sur les schémas les plus longs.

Conclusion. *Guidance* apparaît comme plus fiable pour garantir la conformité finale dans nos conditions expérimentales. À l'inverse, *Outlines* met davantage en évidence les limites pratiques d'une approche fondée sur les automates finis face à l'expressivité de *JSON Schema*.

Le choix entre les deux dépend donc du compromis recherché : une meilleure couverture des schémas complexes pour *Guidance*, ou davantage de simplicité et de prévisibilité pour *Outlines* sur un périmètre de schémas réguliers.

3.4.3 Tableau récapitulatif

TABLE 3.3 – Comparaison des performances et des comportements des frameworks.

Critère	**LLM libre**	**Outlines**	**Guidance**
Taux de succès global	0,4 %	53,8 %	80 % (estimé)
Échecs de compilation	Non applicable	3 770 (39 %)	1 009 (10,6 %)
Échecs de génération	Majorité des sorties dérivent ou sont invalides	297 (3 %)	837 (8 %)
Échecs de validation	Sorties presque toujours non conformes	376 (3,9 %)	58 (0,6 %)
Couverture des features	Aucune garantie formelle	Bonne sur contraintes locales, faible sur opérateurs logiques	Meilleure couverture globale, mais dépendante des versions
Vitesse / TPOT	Référence libre	Plus lent, surtout à cause de la compilation FSM	Meilleur dans l'article grâce au *fast-forwarding*
Style d'échec dominant	Dérive du prompt, JSON invalide, refus	Timeout compilation, features non supportées, OOM	Timeout génération, quelques features refusées

Chapitre 4

Perspectives

4.1 Travaux non réalisés par manque de temps

Les contraintes temporelles et matérielles inhérentes à un projet de Master nous ont conduits à restreindre notre périmètre. Plusieurs pistes prometteuses méritent d'être documentées pour orienter des travaux futurs.

4.1.1 Extension à XGrammar et comparaison à trois

Notre évaluation se limite à Outlines et Guidance, laissant de côté **XGrammar** (MLC), le framework le plus performant selon le papier original en termes de latence. Son architecture bas-niveau (bitmasks CUDA) constitue une direction particulièrement intéressante pour les environnements de production à haut débit. Une campagne comparative à trois termes permettrait de valider si le gain de performance de XGrammar se maintient sur l'ensemble du *JSONSchemaBench*, notamment sur les schémas *Hard* et *Ultra*.

4.1.2 Évaluation du Pipeline A

Faute de disposer d'un accès stable et dédié au GPU, nous n'avons pas pu mener des mesures de latence fiables (GCT, TTFT, TPOT). Un environnement de benchmarking dédié (instance cloud avec GPU réservé, méthode de mesure par événements CUDA) aurait permis de quantifier précisément le surcoût temporel réel de chaque framework et de tracer des courbes *latence vs complexité du schéma*.

4.1.3 Modélisation explicable des causes d'échec

Une piste que nous n'avons pas eu le temps d'explorer consisterait à construire un fichier `CSV` enrichi où chaque schéma serait représenté par un **vecteur de caractéristiques** incluant :

- la taille du schéma et sa profondeur d'imbrication ;
- le nombre de propriétés et la présence de tableaux ou de références ;
- l'utilisation de mots-clés spécifiques tels que `oneOf`, `anyOf`, `not`, `required`, `pattern`, `uniqueItems` ou `dependencies`.

On pourrait ensuite associer à chaque schéma le résultat obtenu avec *Outlines* et *Guidance* (succès, échec de compilation, échec de génération ou échec de validation). Sur cette base, l'utilisation d'un **modèle interprétable** (tel qu'un *arbre de décision*, une *forêt aléatoire* ou *XGBoost*) permettrait d'identifier automatiquement les caractéristiques les plus fortement corrélées aux échecs.

L'objectif ne serait pas seulement de prédire si un framework va réussir, mais surtout de comprendre quelles *features* ou combinaisons de *features* rendent certains schémas particulièrement difficiles à traiter.

4.1.4 Explorer le décodage contraint sous d'autres formes

Les approches étudiées dans ce rapport (*Outlines*, *Guidance*, *XGrammar*) partagent toutes une caractéristique commune : elles opèrent token par token, à un niveau très bas de la génération. Cette granularité fine permet un contrôle strict, mais elle introduit un coût computationnel élevé et se heurte structurellement aux constructions logiques globales de *JSON Schema*.

Une direction de recherche fondamentalement différente consiste à déplacer les contraintes : non plus localement à chaque token, mais globalement sur la séquence entière. Les modèles de diffusion discrets, qui génèrent et raffinent une séquence complète en plusieurs passes plutôt que token par token, offrent un cadre naturel pour ce type de contraintes.

Des constructions comme `uniqueItems`, `dependentRequired` ou `not` (difficiles voire impossibles à encoder dans une machine à états finis) pourraient être intégrées comme des projections sur l'espace des séquences valides à chaque étape de raffinement.

Cette piste, encore exploratoire dans la littérature, fait l'objet d'un stage d'été proposé dans ce cadre, visant à évaluer la faisabilité d'un décodage contraint par diffusion sur des schémas *JSON* complexes.

Chapitre 5

Conclusion des expérimentations

Ce travail avait pour objectif d'étudier le décodage contraint appliqué à la génération de sorties structurées par les *LLMs*, en prenant *JSON Schema* comme langage de spécification et *JSONSchemaBench* comme référence expérimentale.

Limites du prompting seul. La baseline sans framework illustre clairement l'insuffisance du prompting seul : avec le modèle 1B utilisé, seuls 39 schémas sur 9 558 aboutissent à une sortie valide (0,4 %). Les échecs révèlent des dérives typiques (répétition du prompt, copie du schéma, JSON incomplet, ajout de texte naturel) ainsi que des contraintes techniques liées à la fenêtre de contexte sur les schémas volumineux.

Performance des frameworks. L'utilisation de frameworks améliore fortement la situation, sans supprimer toutes les difficultés.

Outlines atteint 54 % de succès global, mais ses échecs se concentrent à la compilation : les opérateurs logiques comme `not`, `oneOf` ou `anyOf` provoquent des explosions combinatoires qui rendent le framework difficilement utilisable sans pré-filtrage des schémas.

Guidance atteint 80 % de succès global et garantit une conformité quasi totale sur les schémas qu'il accepte (seulement 0,8 % d'erreurs de validation sur les outputs rendus) mais au prix d'un refus préventif de nombreuses features et de 836 *timeouts* de génération liés au *template interleaving* token par token sur les schémas longs.

Un paradoxe révélateur. Notre analyse montre que les deux frameworks atteignent un taux de succès post-compilation quasiment identique 88,6 % pour *Outlines* contre 89,5 % pour *Guidance*. L'écart de performance global s'explique donc non pas par une meilleure qualité de génération de *Guidance*, mais par sa capacité à compiler davantage de schémas (8 549 contre 5 827). Ce résultat, absent du papier original, nuance l'interprétation des scores agrégés.

Analyse par feature. L'analyse fondée sur la *JSON Schema Test Suite* révèle que les scores moyens ne suffisent pas à caractériser la fiabilité d'un framework. Il est nécessaire d'identifier précisément quelles contraintes sont supportées, à quelle version, et dans quelles conditions.

Nos résultats mettent en évidence des améliorations silencieuses (des features progressant sans annonce officielle) ainsi que des régressions techniques entre versions, soulignant l'instabilité des moteurs sous-jacents. Une limite méthodologique importante apparaît également : un score positif peut parfois refléter un simple comportement de rejet systématique plutôt qu'un véritable support sémantique de la feature.

Contribution principale. Notre apport réside dans la reconstruction d'une analyse granulaire des causes d'échec (compilation, génération, validation) à l'échelle globale du benchmark, une dimension absente du papier original. Cette décomposition fine, couplée à l'étude des écarts entre

versions, fournit des critères opérationnels plus précis pour choisir et configurer un framework de décodage contraint en conditions réelles.

Conclusion générale. Le décodage contraint est indispensable pour fiabiliser les sorties des *LLMs*, mais ne constitue pas une garantie universelle. Sa réussite dépend du schéma, du framework, de sa version et des ressources disponibles autant de dimensions que les métriques agrégées de l'état de l'art ne permettent pas d'appréhender seules.

Bibliographie

[1] Geng et al. (2025). *JSONSchemaBench : A Comprehensive Benchmark for Constrained Decoding with JSON Schema*. Preprint arXiv.

[2] Willard, B., & Louf, R. (2023). *Efficient Guided Generation for Large Language Models*. arXiv :2307.09702.

[3] Lundberg, S. et al. (2023). *Guidance : A Guidance Language for Controlling Large Language Models*. Microsoft Research. `https://github.com/guidance-ai/guidance`.

[4] Dong, Y. et al. (2024). *XGrammar : Flexible and Efficient Structured Generation Engine for Large Language Models*. arXiv :2411.15100.